



Despliegue de aplicaciones en contenedores DAC

UD3 Contenedores y Docker

Ejercicios obligatorios.....	2
1) Instalación de Docker.....	2
2) Obtención de imágenes y generación de contenedores.....	6
3) Generación de volúmenes.....	9
4) Dockerfile.....	13
Ejercicios opcionales.....	17
Test de autoevaluación.....	17
Instala el Subsystem Linux para Windows o WSL con una máquina Ubuntu y prueba a ejecutar algún comando en ella.....	17
Instala Docker Desktop en Windows (si optas por hacer este ejercicio tendrás que hacer antes el anterior).....	17
Instala Portainer y pruébalo.....	18
Bibliografía web.....	18
Valoración personal.....	19

Juan Cruz, Garrido Suso
Práctica UD3 - Contenedores y Docker
Nombre del módulo: DAM-DAW

Ejercicios obligatorios

1) Instalación de Docker

Usando ssh, instalaremos Docker Engine usando el siguiente script:

```
#!/bin/bash
# sacado de https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Comprobación de su correcta instalación
sudo systemctl status docker containerd
docker --version
# Correr el primer contenedor de prueba
sudo docker run hello-world
```

Este script está disponible en <https://delarioja.org/scripts/installar-docker.sh> , desde ahí lo descargaremos y lo lanzaremos.

<pre>delarioja@ubuntu-juancri:~\$ wget https://delarioja.org/scripts/installar-docker.sh --2023-02-28 12:11:52-- https://delarioja.org/scripts/installar-docker.sh Resolviendo delarioja.org (delarioja.org)... 104.24.24.3, 104.24.24.3, 104.24.10.1, ... Conectando con delarioja.org (delarioja.org)[104.24.24.3]:80... conectado. Envío de HTTP request: expected... 200 OK Longitud: 879 [application/x-sh] Guardando como: 'instalar-docker.sh' instalar-docker.sh 100%[.....] 879 --KB/s en 0s delarioja@ubuntu-juancri:~\$ cat instalar-docker.sh #!/bin/bash # sacado de https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository # Add Docker's official GPG key: sudo apt-get update sudo apt-get install ca-certificates curl sudo install -m 0755 -d /etc/apt/keyrings sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc sudo chmod a+r /etc/apt/keyrings/docker.asc # Add the repository to Apt sources: echo \ "deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \ \$(. /etc/os-release && echo "\$VERSION_CODENAME") stable" \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null sudo apt-get update sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin # Comprobación de su correcta instalación sudo systemctl status docker docker --version # Correr el primer contenedor de prueba sudo docker run hello-world</pre>	<pre>Info: Adding new user 'alumnat' (1885) with group 'alumnat' (1884) ... Info: Creando el directorio personal '/home/alumnat' ... Info: Copiando los ficheros desde '/etc/skel' ... Nueva contraseña: Vuelva a escribir la nueva contraseña: passwd: contraseña actualizada correctamente Cambiando la información de usuario para alumnat Introduzca el nuevo valor, o presione ENTER para el predeterminado Nombre completo []: Número de habitación []: Teléfono del trabajo []: Teléfono de casa []: Otro []: ¿Es correcta la información [5/n] s Info: Adding new user 'alumnat' to supplemental / extra groups 'users' ... Info: Afiliando al usuario 'alumnat' al grupo 'users' ... Integración de usuarios: s suma adminer Alumnat Info: Modificando el usuario 'alumnat' ... Info: Selecting UID/GID from range 1000 to 5000 ... Info: Adding new user 'alumnat' (1885) with group 'alumnat' (1885) ... Info: Creando el directorio personal '/home/alumnat' ... Info: Copiando los ficheros desde '/etc/skel' ... Nueva contraseña: Vuelva a escribir la nueva contraseña: passwd: contraseña actualizada correctamente Cambiando la información de usuario para alumnat Introduzca el nuevo valor, o presione ENTER para el predeterminado Nombre completo []: Número de habitación []: Teléfono del trabajo []: Teléfono de casa []: Otro []: ¿Es correcta la información [5/n] s Info: Adding new user 'alumnat' to supplemental / extra groups 'users' ... Info: Afiliando al usuario 'alumnat' al grupo 'users' ... delarioja@ubuntu-juancri:~\$</pre>
<pre>wget https://delarioja.org/scripts/installar-docker.sh cat instalar-docker.sh Descarga del script y comprobación de su contenido</pre>	<pre>chmod +x instalar-docker.sh ./instalar-docker.sh El script incluye la comprobación del servicio con systemctl y la solicitud de versión a través del comando docker.</pre>

Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juancruz:~$ docker version
Client: Docker Engine - Community
Version: 28.0.0
API version: 1.48
Go version: go1.23.6
Git commit: f9ced58
Built: Wed Feb 19 22:11:04 2025
OS/Arch: linux/amd64
Context: default
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.48/version": dial unix /var/run/docker.sock: connect: permission denied

delarioja@ubuntu-juancruz:~$ sudo docker version
Client: Docker Engine - Community
Version: 28.0.0
API version: 1.48
Go version: go1.23.6
Git commit: f9ced58
Built: Wed Feb 19 22:11:04 2025
OS/Arch: linux/amd64
Context: default

Server: Docker Engine - Community
Engine:
Version: 28.0.0
API version: 1.48 (minimum version 1.24)
Go version: go1.23.6
Git commit: af898ab
Built: Wed Feb 19 22:11:04 2025
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: 1.7.25
GitCommit: bcc810d6b9066471b0b6fa75f557a15a1cbf31bb
runc:
Version: 1.2.4
GitCommit: v1.2.4-0-g6c52b3f
docker-init:
Version: 0.19.0
GitCommit: de40ad0

delarioja@ubuntu-juancruz:~$ sudo docker run hello-world
[sudo] contraseña para delarioja:
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
e6590344b1a5: Pull complete
Digest: sha256:e0b569a5163a5e6be84e210a2587e7d447e08f87a0e90798363fa44a0464a1e8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

docker version

Compruebo que la versión que he instalado es diferente a la de clase (esta es la 28) y además aparece un aviso de que no parece estar disponible el socket para conectar.

Al hacerlo con sudo (sudo docker version), sí parece funcionar.

Así, el sudo docker run hello-world funciona correctamente.



Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

Cargar los perfiles personales y de sistema tardó 693ms.
(base) PS C:\Users\jcgsu> ssh delarioja@ubuntu-juancruz
delarioja@ubuntu-juancruz's password:
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-45-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

El mantenimiento de seguridad expandido para Applications está desactivado

Se pueden aplicar 350 actualizaciones de forma inmediata.
128 de estas son actualizaciones de seguridad estándares.
Para ver estas actualizaciones adicionales, ejecute: apt list --upgradable

Active ESM Apps para recibir futuras actualizaciones de seguridad adicionales.
Vea https://ubuntu.com/esm o ejecute «sudo pro status»

Last login: Mon Feb 24 11:47:42 2025 from 192.168.68.102
delarioja@ubuntu-juancruz:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
delarioja@ubuntu-juancruz:~$
```

Por último, añadimos nuestro usuario al grupo docker de modo que podamos ejecutarlo sin sudo.

```
sudo adduser delarioja docker
```

(en consola y conectado por ssh, he necesitado relogarme para que tenga efecto y lo siguiente funcione)

```
docker ps
```


Juan Cruz Garrido Suso
Práctica Unidad 3
Despliegue de aplicaciones en contenedores

2) Obtención de imágenes y generación de contenedores

En primer lugar, buscamos y descargamos las imágenes de MariaDB y de Joomla.

```
circleci/mariadb          CircleCI images for MariaDB          4
bitnamicharts/mariadb     Bitnami Helm chart for MariaDB          0
linuxserver/mariadb       A Mariadb container, brought to you by Linux... 403
cimg/mariadb              0
elestio/mariadb           Mariadb, verified and packaged by Elestio 1
docksal/mariadb           MariaDB service images for Docksal - https:/... 0
chainguard/mariadb       Build, ship and run secure software with Cha... 0
uselagoon/mariadb        0
jumpserver/mariadb       MariaDB Server is a high performing open sou... 0
islandora/mariadb        Open source relational database.         0
corpusops/mariadb        https://github.com/corpusops/docker-images/ 0
alpinelinux/mariadb      0
reseaucerta/mariadb      0
encoflife/mariadb        0
alpine/mariadb           2
delarioja@ubuntu-juancruz:~$ docker pull mariadb
Using default tag: latest
latest: Pulling from library/mariadb
5a7813e071bf: Pull complete
bdec990c29c: Pull complete
5db80086e4da: Pull complete
901fe9394c00: Pull complete
43eb19e1b102: Pull complete
597f7afe50fe: Pull complete
e1dede558384: Pull complete
5c3a22df929b: Pull complete
Digest: sha256:310d29fbb58169dcddb384b0ff138edb081e2773d6e2eceb976b3668089f2f84
Status: Downloaded newer image for mariadb:latest
docker.io/library/mariadb:latest
delarioja@ubuntu-juancruz:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED   STATUS    PORTS   NAMES
delarioja@ubuntu-juancruz:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED      SIZE
joomla        latest   5bb8344947c8   5 days ago   756MB
mariadb       latest   a914eff5d2eb   10 days ago   336MB
hello-world   latest   74cc54e27dc4   4 weeks ago   10.1kB
delarioja@ubuntu-juancruz:~$
```

```
docker search joomla
docker search mariadb
```

```
docker pull [las dos]
```

```
docker images
```

Buscamos, descargamos y comprobamos que las imágenes están ya disponibles en local.

Ahora es necesario crear una red que permitirá comunicarse a los dos contenedores que levantaremos.

```
delarioja@ubuntu-juancruz:~$ docker network create joomla-network
059edb94fc74111acd3d83e4180a85e21eb14457b26eae0477355ae17cf0d4
```

Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juancruz:~$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
0ff0921fc56f        bridge             bridge              local
f39f92c2dfef        host               host                local
059edb94fc74        joomla-network     bridge              local
08c9a39cc680        none               null                local
```

Red creada y vista en la lista.

```
docker network create joomla-network
docker network ls
```

Ahora ya podemos lanzar los contenedores, para ello ejecutaremos los siguientes comandos:

```
docker run --rm -d --name mariadb \
-e MYSQL_ROOT_PASSWORD=org \
-e MYSQL_DATABASE=joomla \
-e MYSQL_USER=joomlauser \
-e MYSQL_PASSWORD=org \
--network joomla-network \
mariadb:latest
```

```
docker run --rm -d --name joomla \
-p 8088:80 \
-e JOOMLA_DB_HOST=mariadb \
-e JOOMLA_DB_USER=joomlauser \
-e JOOMLA_DB_PASSWORD=org \
-e JOOMLA_DB_NAME=joomla \
--network joomla-network \
joomla:latest
```

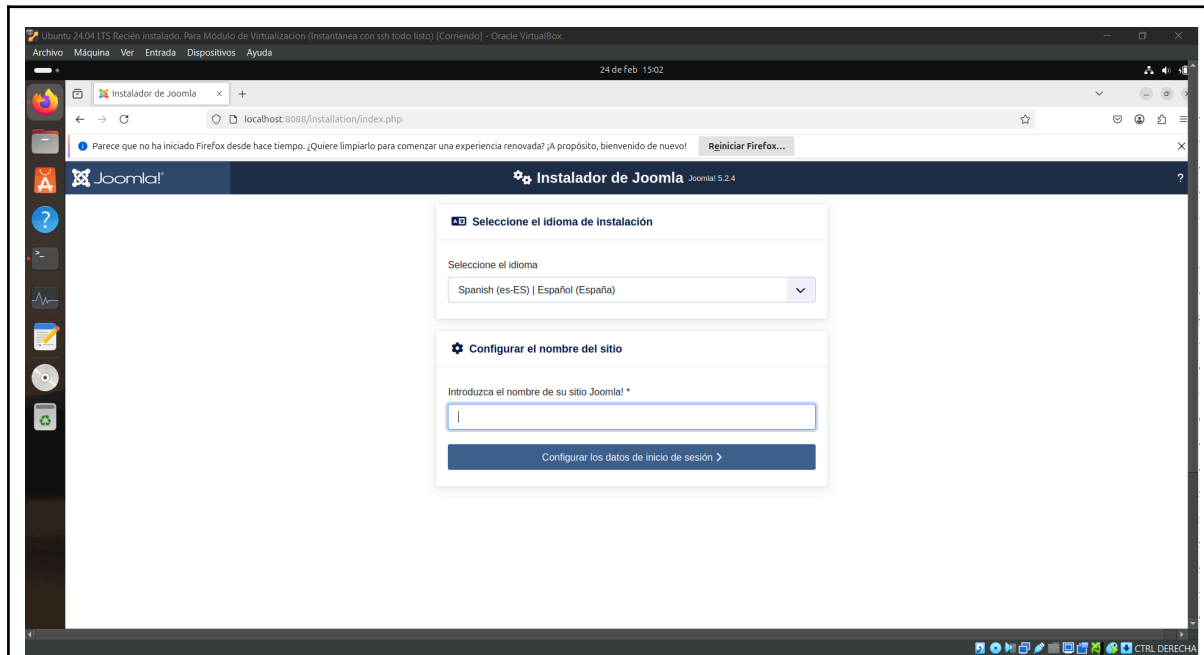
Como vemos, en estos comandos se precisan usuarios y nombres de recursos, en particular el servicio de bbdd, el nombre de la bbdd, el usuario, password etc.

Me parece importante señalar que -p xxxx:xxxx indicará el redireccionamiento de puertos, de modo que el primer número es el puerto en el anfitrión y el segundo en el contenedor.

Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores



Como se puede comprobar, joomla ya está funcionando a través de un contenedor en nuestro ubuntu.

Por último comprobamos lo que nos dice docker sobre sus contenedores:

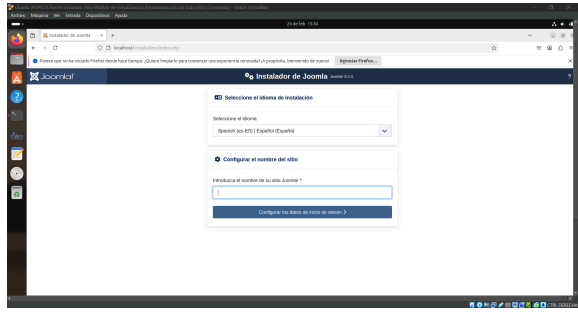
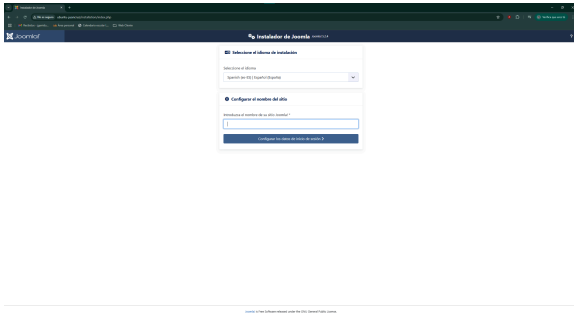
```
delarioja@ubuntu-juancruz:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
7228acc4aa79   joomla:latest  "/entrypoint.sh apac..." 23 minutes ago Up 23 minutes 0.0.0.0:8088->80/tcp, [::]:8088->80/tcp   joomla
a0b33d4a4bc4   mariadb:latest "docker-entrypoint.s..." 23 minutes ago Up 23 minutes 3306/tcp                               mariadb
```

Podemos comprobar como constan los dos levantados y con detalles sobre su configuración, en particular la imagen en que se basan y el redireccionamiento de puertos.

```
docker ps
```

Nota: al intentar acceder desde el windows anfitrión del ubuntu no me deja acceder, así que he detenido el contenedor de joomla y lo he lanzado sobre el puerto 80, y ahora sí, también me deja acceder desde el windows que los aloja a todos.

Juan Cruz Garrido Suso
Práctica Unidad 3
Despliegue de aplicaciones en contenedores

Joomla corriendo sobre el puerto 80 visto desde ubuntu y desde windows.

```
docker stop joomla
```

```
docker run --rm -d --name joomla
-p 80:80
-e JOOMLA_DB_HOST=mariadb
-e JOOMLA_DB_USER=joomlauser
-e JOOMLA_DB_PASSWORD=org
-e JOOMLA_DB_NAME=joomla
--network joomla-network
joomla:latest
```

3) Generación de volúmenes

Un volumen en Docker es un recurso que nos permitirá tener persistencia a través de un sistema de archivos que pertenece al host y es, por tanto, accesible tanto por él como por los contenedores que le asociemos, en particular la siguiente vez que levantemos el contenedor, lo que le proporciona persistencia entre sesiones.

Empezamos visualizando los volúmenes gestionados por Docker.

```
delarioja@ubuntu-juancruz:~$ docker volume ls
DRIVER      VOLUME NAME
local       1a26370965b0f10d10cdc671df496484f80a4a32364274d0859febaad7ddd7cc
local       2983011d9c4aa3b294dbb29a9921ad6b3de377bf17a6d891188add863759324f
delarioja@ubuntu-juancruz:~$ docker volume create joomla_data
docker volume create mariadb_data
joomla_data
mariadb_data
delarioja@ubuntu-juancruz:~$ docker volume ls
DRIVER      VOLUME NAME
local       1a26370965b0f10d10cdc671df496484f80a4a32364274d0859febaad7ddd7cc
local       2983011d9c4aa3b294dbb29a9921ad6b3de377bf17a6d891188add863759324f
local       joomla_data
local       mariadb_data
delarioja@ubuntu-juancruz:~$ |
```


Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

```
docker volume ls
```

Vemos que actualmente tenemos dos volúmenes, que deben ser volúmenes montados por las imágenes de Joomla y de mariadb. Creamos como se nos pide un volumen para cada contenedor y volvemos a listar los contenedores.

```
docker volume create joomla_data
docker volume create mariadb_data
```

Ahora inspeccionaremos esos volúmenes a ver qué nos dice Docker

```
delarioja@ubuntu-juancruz:~$ docker volume ls
DRIVER      VOLUME NAME
local       1a26370965b0f10d10cdc671df496484f80a4a32364274d0859febaad7ddd7cc
local       2983011d9c4aa3b294dbb29a9921ad6b3de377bf17a6d891188add863759324f
local       joomla_data
local       mariadb_data
delarioja@ubuntu-juancruz:~$ docker inspect joomla_data
[
  {
    "CreatedAt": "2025-02-24T17:57:32+01:00",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/joomla_data/_data",
    "Name": "joomla_data",
    "Options": null,
    "Scope": "local"
  }
]
delarioja@ubuntu-juancruz:~$ docker inspect 1a26370965b0f10d10cdc671df496484f80a4a32364274d0859febaad7ddd7cc
[
  {
    "CreatedAt": "2025-02-24T17:50:42+01:00",
    "Driver": "local",
    "Labels": {
      "com.docker.volume.anonymous": ""
    },
    "Mountpoint": "/var/lib/docker/volumes/1a26370965b0f10d10cdc671df496484f80a4a32364274d0859febaad7ddd7cc/_data",
    "Name": "1a26370965b0f10d10cdc671df496484f80a4a32364274d0859febaad7ddd7cc",
    "Options": null,
    "Scope": "local"
  }
]
delarioja@ubuntu-juancruz:~$
```

```
docker inspect joomla_data
```

Podemos comprobar como con ese comando se nos informa entre otras cosas del punto de montaje, que tiene como raíz /var/lib/docker/volumes

```
delarioja@ubuntu-juancruz:~$ sudo ls /var/lib/docker/volumes
1a26370965b0f10d10cdc671df496484f80a4a32364274d0859febaad7ddd7cc  backingFsBlockDev  mariadb_data
2983011d9c4aa3b294dbb29a9921ad6b3de377bf17a6d891188add863759324f  joomla_data        metadata.db
delarioja@ubuntu-juancruz:~$ sudo ls -l /var/lib/docker/volumes
total 40
drwx-----x 3 root root 4096 feb 24 17:50 1a26370965b0f10d10cdc671df496484f80a4a32364274d0859febaad7ddd7cc
drwx-----x 3 root root 4096 feb 24 14:58 2983011d9c4aa3b294dbb29a9921ad6b3de377bf17a6d891188add863759324f
brw----- 1 root root 8, 1 feb 24 12:19 backingFsBlockDev
drwx-----x 3 root root 4096 feb 24 17:57 joomla_data
drwx-----x 3 root root 4096 feb 24 17:57 mariadb_data
-rw----- 1 root root 32768 feb 24 17:57 metadata.db
delarioja@ubuntu-juancruz:~$ sudo ls -l /var/lib/docker/volumes/mariadb_data
total 4
drwxr-xr-x 2 root root 4096 feb 24 17:57 _data
delarioja@ubuntu-juancruz:~$
```

Aquí vemos esos contenidos (ojo, hay que acceder con sudo)

```
sudo ls -l /var/lib/docker/volumes
```

Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

Ahora, se nos pide asociar esos volúmenes a los contenedores correspondientes, para ello he detenido la ejecución de los contenedores primero y vuelto a lanzar, incluyendo en este caso la asociación pedida (-v):

```
docker run --rm -d --name mariadb \  
  -e MYSQL_ROOT_PASSWORD=org \  
  -e MYSQL_DATABASE=joomla \  
  -e MYSQL_USER=joomlauser \  
  -e MYSQL_PASSWORD=org \  
  --network joomla-network \  
  -v mariadb_data:/var/lib/mysql \  
  mariadb:latest
```

```
docker run --rm -d --name joomla \  
  -p 8088:80 \  
  -e JOOMLA_DB_HOST=mariadb \  
  -e JOOMLA_DB_USER=joomlauser \  
  -e JOOMLA_DB_PASSWORD=org \  
  -e JOOMLA_DB_NAME=joomla \  
  --network joomla-network \  
  -v joomla_data:/var/www/html \  
  joomla:latest
```

```
delarioja@ubuntu-juancruz:~$ docker stop joomla  
joomla  
delarioja@ubuntu-juancruz:~$ docker stop mariadb  
mariadb  
delarioja@ubuntu-juancruz:~$ docker run --rm -d --name mariadb \  
  -e MYSQL_ROOT_PASSWORD=org \  
  -e MYSQL_DATABASE=joomla \  
  -e MYSQL_USER=joomlauser \  
  -e MYSQL_PASSWORD=org \  
  --network joomla-network \  
  -v mariadb_data:/var/lib/mysql \  
  mariadb:latest  
bfff2d681dd865609137e4cff66a2741d90c5ac7e64af3f31a77e1f0142413e2b  
delarioja@ubuntu-juancruz:~$ docker run --rm -d --name joomla \  
  -p 8088:80 \  
  -e JOOMLA_DB_HOST=mariadb \  
  -e JOOMLA_DB_USER=joomlauser \  
  -e JOOMLA_DB_PASSWORD=org \  
  -e JOOMLA_DB_NAME=joomla \  
  --network joomla-network \  
  -v joomla_data:/var/www/html \  
  joomla:latest  
c3b205c12d797348bd73e04a49b00f54b63dfa9b2e5d3ca607946f139ee2c915  
delarioja@ubuntu-juancruz:~$
```

Contenedores detenidos y vueltos a lanzar con el nuevo volumen.

Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juancruz:~$ docker exec -it joomla /bin/bash
root@c3b205c12d79:/var/www/html# mkdir directorio_prueba
root@c3b205c12d79:/var/www/html# exit
exit
delarioja@ubuntu-juancruz:~$ sudo ls -l /var/lib/docker/volumes/joomla_data
[sudo] contraseña para delarioja:
total 4
drwxr-xr-x 19 www-data www-data 4096 feb 25 09:59 _data
delarioja@ubuntu-juancruz:~$ sudo ls -l /var/lib/docker/volumes/joomla_data/_data
total 116
drwxr-xr-x 11 www-data www-data 4096 feb 4 11:42 administrator
drwxr-xr-x 5 www-data www-data 4096 feb 4 11:42 api
drwxr-xr-x 2 www-data www-data 4096 feb 4 11:42 cache
drwxr-xr-x 2 www-data www-data 4096 feb 4 11:42 cli
drwxr-xr-x 18 www-data www-data 4096 feb 4 11:42 components
drwxr-xr-x 2 root root 4096 feb 25 09:59 directorio_prueba ←
-rw-r--r-- 1 www-data www-data 6899 feb 4 11:42 htaccess.txt
drwxr-xr-x 5 www-data www-data 4096 feb 4 11:42 images
drwxr-xr-x 2 www-data www-data 4096 feb 4 11:42 includes
-rw-r--r-- 1 www-data www-data 1060 feb 4 11:42 index.php
drwxr-xr-x 10 www-data www-data 4096 feb 4 11:42 installation
drwxr-xr-x 4 www-data www-data 4096 feb 4 11:42 language
drwxr-xr-x 6 www-data www-data 4096 feb 4 11:42 layouts
drwxr-xr-x 6 www-data www-data 4096 feb 4 11:43 libraries
-rw-r--r-- 1 www-data www-data 18092 feb 4 11:42 LICENSE.txt
drwxr-xr-x 76 www-data www-data 4096 feb 4 11:44 media
drwxr-xr-x 27 www-data www-data 4096 feb 4 11:42 modules
drwxr-xr-x 26 www-data www-data 4096 feb 4 11:42 plugins
-rw-r--r-- 1 www-data www-data 5034 feb 4 11:42 README.txt
-rw-r--r-- 1 www-data www-data 764 feb 4 11:42 robots.txt.dist
drwxr-xr-x 4 www-data www-data 4096 feb 4 11:42 templates
drwxr-xr-x 2 www-data www-data 4096 feb 4 11:42 tmp
-rw-r--r-- 1 www-data www-data 2974 feb 4 11:42 web.config.txt
delarioja@ubuntu-juancruz:~$ |
```

Comprobación de que en efecto el volumen que ve Docker y nuestro sistema de archivos coinciden.

1- Accedemos al sistema de archivos del contenedor joomla con:

```
docker exec -it joomla /bin/bash
```

2- Creamos un directorio de prueba y salimos:

```
mkdir directorio_prueba
```

```
exit
```

3- Ya desde el host accedemos al directorio a ver qué tenemos con:

```
sudo ls -l /var/lib/docker/volumes/joomla_data/_data
```

Vemos que accedemos y el directorio se ha creado con propietario root



Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

4) Dockerfile

Un Dockerfile es un archivo que nos permitirá construir imágenes de Docker según nuestras necesidades. Se parte de una imagen dada y se pueden ejecutar una serie de instrucciones sobre ella que configurarán el entorno de ejecución del contenedor.

En nuestro escenario la idea es desarrollar Dockerfiles que reflejen la configuración que estábamos haciendo con `docker run`.

El Dockerfile para Joomla me queda así:

```
FROM joomla:latest

ENV JOOMLA_DB_HOST=mariadb
ENV JOOMLA_DB_USER=joomlauser
ENV JOOMLA_DB_PASSWORD=org
ENV JOOMLA_DB_NAME=joomla

VOLUME joomla_data:/var/www/html
```

Y para MariaDB:

```
FROM mariadb:latest

ENV MYSQL_ROOT_PASSWORD=org
ENV MYSQL_DATABASE=joomla
ENV MYSQL_USER=joomlauser
ENV MYSQL_PASSWORD=org

VOLUME mariadb_data:/var/lib/mysql
```


Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juancruz:~$ ls
Descargas  Escritorio  instalar-docker.sh  permisosBorra.sh  permisosRecrea.sh  Público  Videos
Documentos  Imágenes  Música  permisosCrea.sh  Plantillas  snap
delarioja@ubuntu-juancruz:~$ mkdir dockerfiles
delarioja@ubuntu-juancruz:~$ ls
Descargas  Documentos  Imágenes  Música  permisosCrea.sh  Plantillas  snap
dockerfiles  Escritorio  instalar-docker.sh  permisosBorra.sh  permisosRecrea.sh  Público  Videos
delarioja@ubuntu-juancruz:~$ cd dockerfiles
delarioja@ubuntu-juancruz:~/dockerfiles$ cd joomla
delarioja@ubuntu-juancruz:~/dockerfiles/joomla$ nano dockerfile
delarioja@ubuntu-juancruz:~/dockerfiles/joomla$ ls
Dockerfile
delarioja@ubuntu-juancruz:~/dockerfiles/joomla$ cat Dockerfile
FROM joomla:latest

ENV JOOMLA_DB_HOST=mariadb
ENV JOOMLA_DB_USER=joomlauser
ENV JOOMLA_DB_PASSWORD=org
ENV JOOMLA_DB_NAME=joomla

VOLUME joomla_data:/var/www/html
delarioja@ubuntu-juancruz:~/dockerfiles/joomla$ docker build -t mi-joomla .
2025/02/25 13:41:33 an: [string]
2025/02/25 13:41:33 Parsed entitlements: []
[+] Building 1.0s (5/5) FINISHED
=> [internal] load build definition from Dockerfile                                docker:default
=> => transferring dockerfile: 203B                                              0.1s
=> [internal] load metadata for docker.io/library/joomla:latest                  0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [1/1] FROM docker.io/library/joomla:latest                                  0.6s
=> exporting layers                                                             0.0s
=> exporting image                                                              0.0s
=> writing image sha256:b9f0ea49db28812088e696c485d86b1b744cb658387a6b9be63dc22525d0d641 0.0s
=> naming to docker.io/library/mi-joomla                                       0.0s

1 warning found (use docker --debug to expand):
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "JOOMLA_DB_PASSWORD") (line 5)
delarioja@ubuntu-juancruz:~/dockerfiles/joomla$ ls
Dockerfile
delarioja@ubuntu-juancruz:~/dockerfiles/joomla$ docker run --rm -d --name joomla -p 8088:80 --network joomla-network mi-joomla
```

Creación del Dockerfile para Joomla, construcción de la imagen y ejecución del contenedor.

Observamos un aviso sobre que no es buena idea establecer variables de entorno relacionadas con datos sensibles como passwords en el dockerfile.

```
docker build -t mi-joomla .
docker run --rm -d --name joomla -p 8088:80 --network joomla-network mi-joomla
```



Joomla ejecutándose correctamente desde nuestra imagen creada según el Dockerfile.



Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

```

delarioja@ubuntu-juancruz:~/dockerfiles/joomla$ cd ..
delarioja@ubuntu-juancruz:~/dockerfiles$ mkdir mariadb
delarioja@ubuntu-juancruz:~/dockerfiles$ cd mariadb/
delarioja@ubuntu-juancruz:~/dockerfiles/mariadb$ nano Dockerfile
delarioja@ubuntu-juancruz:~/dockerfiles/mariadb$ docker build -t mi-mariadb .
2025/02/25 14:02:38 In: [string]
2025/02/25 14:02:38 Parsed entitlements: []
[*] Building 0.5s (5/5) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 399B
=> WARN: SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MYSQL_ROOT_PASSWORD") (line 3)
=> WARN: SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MYSQL_PASSWORD") (line 6)
=> [internal] load metadata for docker.io/library/mariadb:latest
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/1] FROM docker.io/library/mariadb:latest
=> exporting to image
=> exporting layers
=> writing image sha256:cd4ac8cd66523367737828715394de0867c27a19ba9c57b78886c51bf14b1e2
=> naming to docker.io/library/mi-mariadb
2 warnings found (use docker --debug to expand):
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MYSQL_ROOT_PASSWORD") (line 3)
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MYSQL_PASSWORD") (line 6)
delarioja@ubuntu-juancruz:~/dockerfiles/mariadb$ docker run --rm -d --name mariadb --network joomla-network mi-mariadb
6a9afdf7d31295628587ced6aa94b62be89f83a4a45433f8985ef1d92dd83635
delarioja@ubuntu-juancruz:~/dockerfiles/mariadb$

```

Construcción de la imagen para MariaDB y lanzamiento correcto. Mismos avisos.

Llegado el momento de volver a comprobar la persistencia de los volúmenes, compruebo que la instrucción: `VOLUME mariadb_data:/var/lib/mysql` no se ha seguido correctamente y que el contenedor está usando un volumen anónimo, montado en su lugar, pero ignorando el volumen precisado.

Al parecer, la instrucción `VOLUME` nos permitirá crear imágenes, pero si no lo especificamos en el `run`, entonces serán siempre anónimas, es decir, se persistirá el estado del contenedor, pero no se usará un volumen específico no anónimo, aunque la sintaxis del `Dockerfile` nos deje ponerlo y no se queje ni avise de que no tiene efecto.

Por tanto, debo especificar el nombre del volumen con la opción `-v` del comando `docker run`.

```
docker run --rm -d --name joomla -p 8088:80 --network joomla-network -v
joomla_data:/var/www/html mi-joomla
```

```
docker run --rm -d --name mariadb --network joomla-network -v
mariadb_data:/var/lib/mysql mi-mariadb
```

La diferencia entre poner o no poner la etiqueta `VOLUME` en el `Dockerfile` es que al ponerla aseguramos que esos datos quedan fuera y además se declara la intención de usar un volumen para esos directorios, de todos modos toda esa gestión puede hacerse desde el `run`.



Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

```
delarioja@ubuntu-juncruz:~/dockerfiles/mariadb$ docker stop joomla
joomla
delarioja@ubuntu-juncruz:~/dockerfiles/mariadb$ docker run --rm -d --name joomla -p 8088:80 --network joomla-network -v joomla_data:/var/www/html mi-joomla
8c6c13f4d688a3b70b0a5d7ec3c87976f1a2d41e7613d0e472c33f8d49
delarioja@ubuntu-juncruz:~/dockerfiles/mariadb$ docker run --rm -d --name mariadb --network joomla-network -v mariadb_data:/var/lib/mysql mi-mariadb
163780c1b5794187dc7bd3e5a7fed2c2ea26c09fd1b549fd54639fb184e2c3c
delarioja@ubuntu-juncruz:~/dockerfiles/mariadb$ docker exec -it joomla /bin/bash
root@8c6c13f4d688:/var/www/html# ls
LICENSE.txt  administrator  cache  components  htaccess.txt  includes  installation  layouts  media  plugins  templates  web.config.txt
README.txt  api  cli  directorio_prueba  images  index.php  language  libraries  modules  robots.txt.dist  tmp
root@8c6c13f4d688:/var/www/html# ls -l
total 116
-rw-r--r-- 1 www-data www-data 18092 Feb  4 10:42 LICENSE.txt
-rw-r--r-- 1 www-data www-data 5036 Feb  4 10:42 README.txt
drwxr-xr-x 11 www-data www-data 4096 Feb  4 10:42 administrator
drwxr-xr-x 5 www-data www-data 4096 Feb  4 10:42 api
drwxr-xr-x 2 www-data www-data 4096 Feb  4 10:42 cache
drwxr-xr-x 2 www-data www-data 4096 Feb  4 10:42 cli
drwxr-xr-x 18 www-data www-data 4096 Feb  4 10:42 components
drwxr-xr-x 2 root root 4096 Feb 25 08:59 directorio_prueba
-rw-r--r-- 1 www-data www-data 6099 Feb  4 10:42 htaccess.txt
drwxr-xr-x 2 www-data www-data 4096 Feb  4 10:42 images
drwxr-xr-x 2 www-data www-data 4096 Feb  4 10:42 includes
-rw-r--r-- 1 www-data www-data 1060 Feb  4 10:42 index.php
drwxr-xr-x 10 www-data www-data 4096 Feb  4 10:42 installation
drwxr-xr-x 4 www-data www-data 4096 Feb  4 10:42 language
drwxr-xr-x 4 www-data www-data 4096 Feb  4 10:42 layouts
drwxr-xr-x 4 www-data www-data 4096 Feb  4 10:42 libraries
drwxr-xr-x 76 www-data www-data 4096 Feb  4 10:42 media
drwxr-xr-x 27 www-data www-data 4096 Feb  4 10:42 modules
drwxr-xr-x 26 www-data www-data 4096 Feb  4 10:42 plugins
-rw-r--r-- 1 www-data www-data 764 Feb  4 10:42 robots.txt.dist
drwxr-xr-x 4 www-data www-data 4096 Feb  4 10:42 templates
drwxr-xr-x 2 www-data www-data 4096 Feb  4 10:42 tmp
-rw-r--r-- 1 www-data www-data 2074 Feb  4 10:42 web.config.txt
root@8c6c13f4d688:/var/www/html# mkdir directorio_de_prueba_2
root@8c6c13f4d688:/var/www/html# ls
LICENSE.txt  administrator  cache  components  directorio_prueba  images  index.php  language  libraries  modules  robots.txt.dist  tmp
README.txt  api  cli  directorio_de_prueba_2  htaccess.txt  includes  installation  layouts  media  plugins  templates  web.config.txt
exit
delarioja@ubuntu-juncruz:~/dockerfiles/mariadb$ sudo ls -l /var/lib/docker/volumes/joomla_data/_data
[sudo] contraseña para delarioja:
total 128
drwxr-xr-x 11 www-data www-data 4096 Feb  4 11:42 administrator
drwxr-xr-x 5 www-data www-data 4096 Feb  4 11:42 api
drwxr-xr-x 2 www-data www-data 4096 Feb  4 11:42 cache
drwxr-xr-x 2 www-data www-data 4096 Feb  4 11:42 cli
drwxr-xr-x 18 www-data www-data 4096 Feb  4 11:42 components
drwxr-xr-x 2 root root 4096 Feb 25 14:43 directorio_de_prueba_2
drwxr-xr-x 2 root root 4096 Feb 25 09:59 directorio_prueba
-rw-r--r-- 1 www-data www-data 6099 Feb  4 11:42 htaccess.txt
drwxr-xr-x 5 www-data www-data 4096 Feb  4 11:42 images
drwxr-xr-x 2 www-data www-data 4096 Feb  4 11:42 includes
-rw-r--r-- 1 www-data www-data 1060 Feb  4 11:42 index.php
drwxr-xr-x 10 www-data www-data 4096 Feb  4 11:42 installation
drwxr-xr-x 4 www-data www-data 4096 Feb  4 11:42 language
drwxr-xr-x 4 www-data www-data 4096 Feb  4 11:42 layouts
drwxr-xr-x 4 www-data www-data 18092 Feb  4 11:42 LICENSE.txt
drwxr-xr-x 76 www-data www-data 4096 Feb  4 11:42 media
drwxr-xr-x 27 www-data www-data 4096 Feb  4 11:42 modules
drwxr-xr-x 26 www-data www-data 4096 Feb  4 11:42 plugins
-rw-r--r-- 1 www-data www-data 5036 Feb  4 11:42 README.txt
```

Comprobación de la creación del nuevo directorio, podemos comprobar también que el anterior sigue ahí a pesar de haber sido usado por un contenedor basado en una imagen diferente.

Juan Cruz Garrido Suso
Práctica Unidad 3
Despliegue de aplicaciones en contenedores

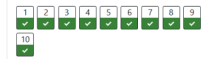
Ejercicios opcionales

Test de autoevaluación

Comenzado el	martes, 25 de febrero de 2025, 14:58
Estado	Finalizado
Finalizado en	martes, 25 de febrero de 2025, 15:00
Tiempo empleado	2 minutos 55 segundos
Calificación	10,00 de 10,00 (100%)

Pregunta 1	¿Cómo se llama al comando que lista todos los contenedores en ejecución?
Correcta	Salvadora suso

Navegación por el cuestionario



Mostrar una página cada vez
Finalizar revisión

Instala el Subsystem Linux para Windows o WSL con una máquina Ubuntu y prueba a ejecutar algún comando en ella.

Ya lo tengo instalado, adjunto pantallazo.

```

delarioja@ubuntu-juancruz: ~/
juan@DESKTOP-FU9HECD: /mr
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

Cargar los perfiles personales y de sistema tardó 800ms.
(base) PS C:\Users\jcgusu> wsl
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/juan/.hushlogin file.
juan@DESKTOP-FU9HECD:/mnt/c/Users/jcgusu$ ll
total 20212
drwxrwxrwx 1 juan juan 4096 Feb 25 11:08 ./
dr-xr-xr-x 1 juan juan 4096 Nov 22 16:06 ../
drwxrwxrwx 1 juan juan 4096 Dec 27 2021 .Origin/
drwxrwxrwx 1 juan juan 4096 Dec 27 2021 .QtWebEngineProcess/
drwxrwxrwx 1 juan juan 4096 Feb 25 11:46 .VirtualBox/
drwxrwxrwx 1 juan juan 4096 Jul 8 2024 .afirma/
drwxrwxrwx 1 juan juan 4096 Jul 16 2021 .anaconda/
drwxrwxrwx 1 juan juan 4096 Jul 15 2021 .astrophy/
drwxrwxrwx 1 juan juan 4096 Dec 19 08:09 .aws/
drwxrwxrwx 1 juan juan 4096 Dec 19 08:09 .azure/
drwxrwxrwx 1 juan juan 4096 Aug 7 2024 .cache/
drwxrwxrwx 1 juan juan 4096 Jul 24 2024 .conda/
-rwxrwxrwx 1 juan juan 25 Jul 16 2021 .condarc*
drwxrwxrwx 1 juan juan 4096 Jul 15 2021 .config/
drwxrwxrwx 1 juan juan 4096 Jul 16 2021 .continuum/
drwxrwxrwx 1 juan juan 4096 Dec 19 08:09 .docker/
drwxrwxrwx 1 juan juan 4096 Jan 23 16:58 .eclipse/
drwxrwxrwx 1 juan juan 4096 Feb 26 2024 .fastai/

```

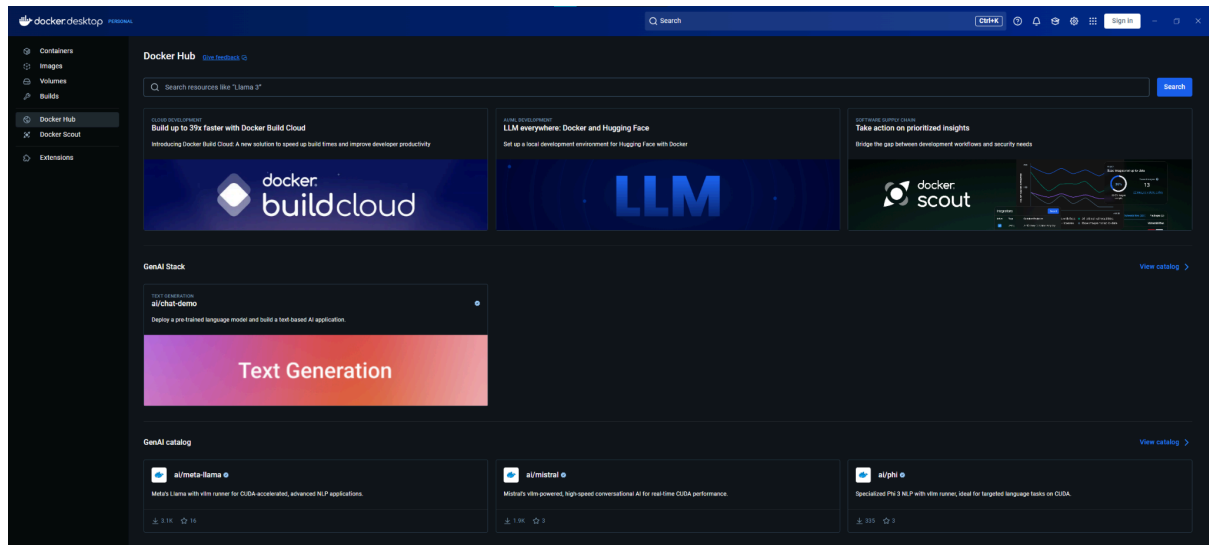
Instala Docker Desktop en Windows (si optas por hacer este ejercicio tendrás que hacer antes el anterior).

También lo tengo instalado.

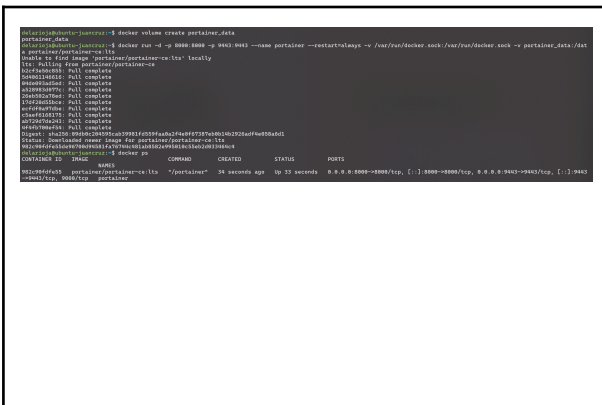
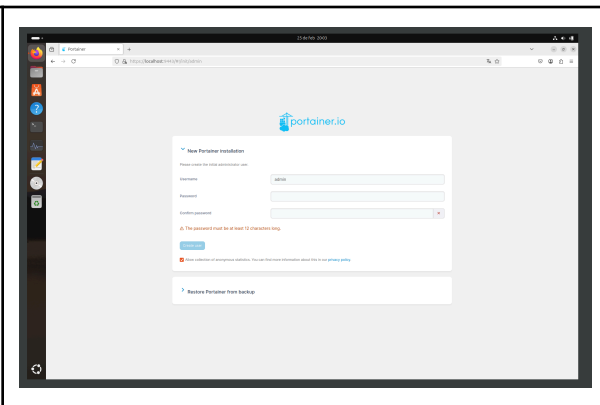
Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores



Instala Portainer y pruébalo.

	
Instalación, siguiendo las instrucciones en https://docs.portainer.io/start/install-ce/server/docker/linux	Instalado y funcionando.

Bibliografía web

- <https://docs.portainer.io/start/install-ce/server/docker/linux>



Juan Cruz Garrido Suso

Práctica Unidad 3

Despliegue de aplicaciones en contenedores

- <https://docs.docker.com/engine/install/ubuntu/>
- <https://claude.ai/> Mi asistente LLM preferido.

Valoración personal

Muy interesante contenido, francamente impresionado con lo que Docker ofrece y lo fácil que resulta montar entornos aislados que garanticen estabilidad, escalabilidad y portabilidad.

Me ha resultado confuso el asunto de los volúmenes anónimos y he estado